Operating Systems Lab Assignment: Synchronization and Scheduling

Your Name

October 24, 2026

1 Introduction

This lab uses threads in C with mutexes and condition variables. I compiled everything with gcc -pthread on Linux.

2 Exercise 1: Hello World

```c
#include <pthread.h>
#include <stdio.h>

pthread_mutex_t lock;
pthread_cond_t cv;
int hello = 0;

void* print_hello(void* arg) {
    pthread_mutex_lock(&lock);
    hello += 1;
    printf("First line (hello=%d)\n", hello);
    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&lock);
    pthread_exit(0);
}

int main() {
    pthread_t thread;
    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    pthread_create(&thread, NULL, print_hello, NULL);
    pthread_mutex_lock(&lock);
    while (hello < 1) {
        pthread_cond_wait(&cv, &lock);
    }
    printf("Second line (hello=%d)\n", hello);
    pthread_mutex_unlock(&lock);

    pthread_join(thread, NULL);
    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}
```

Explanation: The lock and condition variable werent initialized and main didnt wait for the child, so it could print before hello changed. I added the init calls and made main wait in a while loop until hello is 1.

Analysis: The mutex keeps both threads from using hello at the same time and the condition variable makes main wait. Now it always prints in the right order.

Screenshot:



Figure 1: Compilation and execution of hello_world.c

3 Exercise 2: SpaceX Problems

```c
#include <pthread.h>
#include <stdio.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
int n = 3;

void* counter(void* arg) {
    pthread_mutex_lock(&lock);
    while (n > 0) {
        printf("%d\n", n);
        n--;
        pthread_cond_signal(&cv);
        pthread_mutex_unlock(&lock);
        pthread_mutex_lock(&lock);
    }
    pthread_mutex_unlock(&lock);
    return NULL;
}

void* announcer(void* arg) {
    pthread_mutex_lock(&lock);
    while (n != 0) {
        pthread_cond_wait(&cv, &lock);
    }
    printf("FALCON␣HEAVY␣TOUCH␣DOWN!\n");
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, counter, NULL);
    pthread_create(&t2, NULL, announcer, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Explanation: The announcer could print before the countdown was done. Now it waits in a while loop until n is 0 and the counter signals every time it lowers n.

Analysis: The announcer keeps going back to sleep until n is 0, so the message always prints after 1.

Screenshot:

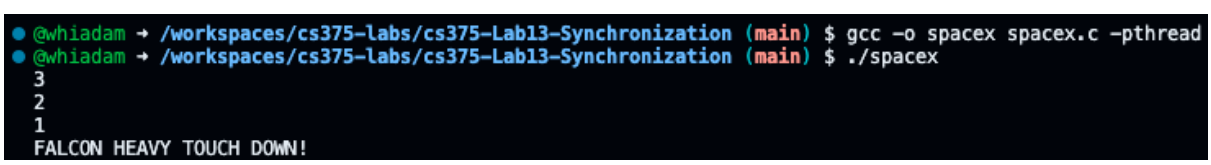


Figure 2: Compilation and execution of spacex.c

4 Exercise 3: I Love You, Unconditionally!

```c
#include <pthread.h>
#include <stdio.h>

pthread_mutex_t lock;
pthread_cond_t cv;
int subaru = 0;

void* helper(void* arg) {
    pthread_mutex_lock(&lock);
    subaru += 1;
    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&lock);
    pthread_exit(0);
}

int main() {
    pthread_t thread;
    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    pthread_create(&thread, NULL, helper, NULL);
    pthread_mutex_lock(&lock);
    while (subaru != 1) {
        pthread_cond_wait(&cv, &lock);
    }
    if (subaru == 1)
        printf("I␣love␣Emilia!\n");
    else
        printf("I␣love␣Rem!\n");
    pthread_mutex_unlock(&lock);

    pthread_join(thread, NULL);
    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}
```

Explanation: Main could check subaru before the helper changed it and print I love Rem. Now main waits in a while loop until subaru is 1.

Analysis: It doesnt matter which thread runs first. The while loop rechecks the condition which is needed for Mesa semantics.

Screenshot:

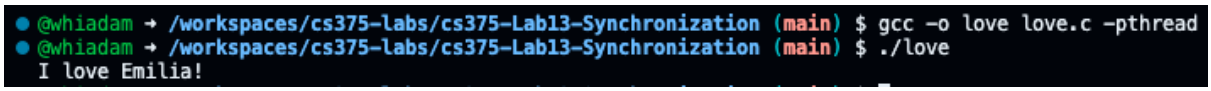


Figure 3: Compilation and execution of love.c

5 Exercise 4: Locking Up the Floopies

```c
#include <pthread.h>
#include <stdio.h>

typedef struct account_t {
    pthread_mutex_t lock;
    int balance;
    long uuid;
```

```c
} account_t;

typedef struct {
    account_t *donor;
    account_t *recipient;
    float amount;
} transfer_args_t;

void transfer(account_t *donor, account_t *recipient, float amount) {
    // Lock accounts in order based on uuid to avoid deadlock
    account_t *first = (donor->uuid < recipient->uuid) ? donor : recipient;
    account_t *second = (donor->uuid < recipient->uuid) ? recipient : donor;

    pthread_mutex_lock(&first->lock);
    pthread_mutex_lock(&second->lock);

    if (donor->balance < amount) {
        printf("Insufficient funds.\n");
    } else {
        donor->balance -= amount;
        recipient->balance += amount;
        printf("Transferred %f from account %ld to %ld\n", amount, donor->uuid,
            recipient->uuid);
    }

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* transfer_thread(void* arg) {
    transfer_args_t *args = (transfer_args_t*)arg;
    transfer(args->donor, args->recipient, args->amount);
    return NULL;
}

int main() {
    account_t acc1 = {PTHREAD_MUTEX_INITIALIZER, 1000, 1};
    account_t acc2 = {PTHREAD_MUTEX_INITIALIZER, 500, 2};

    transfer_args_t args1 = {&acc1, &acc2, 200};
    transfer_args_t args2 = {&acc2, &acc1, 100};

    pthread_t t1, t2;
    pthread_create(&t1, NULL, transfer_thread, &args1);
    pthread_create(&t2, NULL, transfer_thread, &args2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final balance of account %ld: %d\n", acc1.uuid, acc1.balance);
    printf("Final balance of account %ld: %d\n", acc2.uuid, acc2.balance);
    return 0;
}
```

Explanation: Locking donor then recipient can deadlock if two transfers go opposite ways. Now it always locks the lower uuid first. I also added an args struct because the starter code passed the arguments wrong and could crash.

Analysis: Thread 1 locks account 1 and thread 2 locks account 2, then they both wait on each other forever. With the uuid order both threads go for account 1 first so that cant happen. Final balances were 900 and 600.

Screenshot:

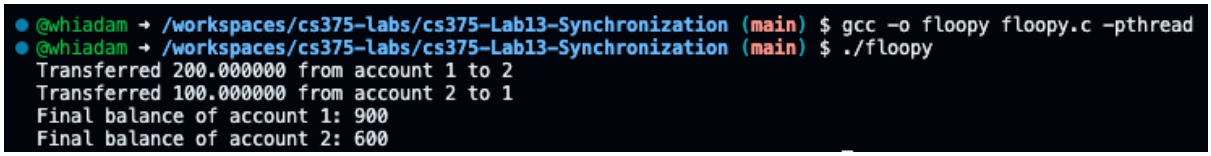


Figure 4: Compilation and execution of floopy.c

6 Exercise 5: Baking with Condition Variables

```c
#include <pthread.h>
#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>

int numBatterInBowl = 0;
int numEggInBowl = 0;
bool readyToEat = false;
int cakesBaked = 0;
pthread_mutex_t lock;
pthread_cond_t needIngredients, readyToBake, startEating;

void addBatter() {
    numBatterInBowl += 1;
    printf("Added batter (batter: %d, eggs: %d)\n", numBatterInBowl,
        numEggInBowl);
}

void addEgg() {
    numEggInBowl += 1;
    printf("Added egg (batter: %d, eggs: %d)\n", numBatterInBowl, numEggInBowl)
        ;
}

void heatBowl() {
    readyToEat = true;
    numBatterInBowl = 0;
    numEggInBowl = 0;
    cakesBaked += 1;
    printf("Baking cake #%d\n", cakesBaked);
}

void eatCake() {
    readyToEat = false;
    printf("Ate cake #%d\n", cakesBaked);
}

void* batterAdder(void* arg) {
    pthread_mutex_lock(&lock);
    while (1) {
        while (numBatterInBowl || readyToEat) {
            pthread_cond_wait(&needIngredients, &lock);
        }
        addBatter();
        pthread_cond_signal(&readyToBake);
        pthread_mutex_unlock(&lock);
        pthread_mutex_lock(&lock);
    }
}

void* eggBreaker(void* arg) {
    pthread_mutex_lock(&lock);
```

```
    while (1) {
        while (numEggInBowl >= 2 || readyToEat) {
            pthread_cond_wait(&needIngredients, &lock);
        }
        addEgg();
        pthread_cond_signal(&readyToBake);
        pthread_mutex_unlock(&lock);
        pthread_mutex_lock(&lock);
    }
}

void* bowlHeater(void* arg) {
    pthread_mutex_lock(&lock);
    while (1) {
        while (numBatterInBowl < 1 || numEggInBowl < 2 || readyToEat) {
            pthread_cond_wait(&readyToBake, &lock);
        }
        heatBowl();
        pthread_cond_signal(&startEating);
        pthread_mutex_unlock(&lock);
        pthread_mutex_lock(&lock);
    }
}

void* cakeEater(void* arg) {
    pthread_mutex_lock(&lock);
    while (1) {
        while (!readyToEat) {
            pthread_cond_wait(&startEating, &lock);
        }
        eatCake();
        pthread_cond_broadcast(&needIngredients);
        pthread_mutex_unlock(&lock);
        pthread_mutex_lock(&lock);
    }
}

int main(int argc, char *argv[]) {
    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&needIngredients, NULL);
    pthread_cond_init(&readyToBake, NULL);
    pthread_cond_init(&startEating, NULL);

    pthread_t batter, egg1, egg2, heater, eater;
    pthread_create(&batter, NULL, batterAdder, NULL);
    pthread_create(&egg1, NULL, eggBreaker, NULL);
    pthread_create(&egg2, NULL, eggBreaker, NULL);
    pthread_create(&heater, NULL, bowlHeater, NULL);
    pthread_create(&eater, NULL, cakeEater, NULL);

    while (1) sleep(1); // Sleep forever

    return 0;
}
```

Explanation: There are threads for batter, eggs, heating and eating that share one lock and three condition variables. I filled in the waits and signals and added some printfs.

Analysis: Nothing can be added while a cake is in the bowl and the heater waits for 1 batter and 2 eggs. It runs forever so I used head to show the first 20 lines.

Screenshot:

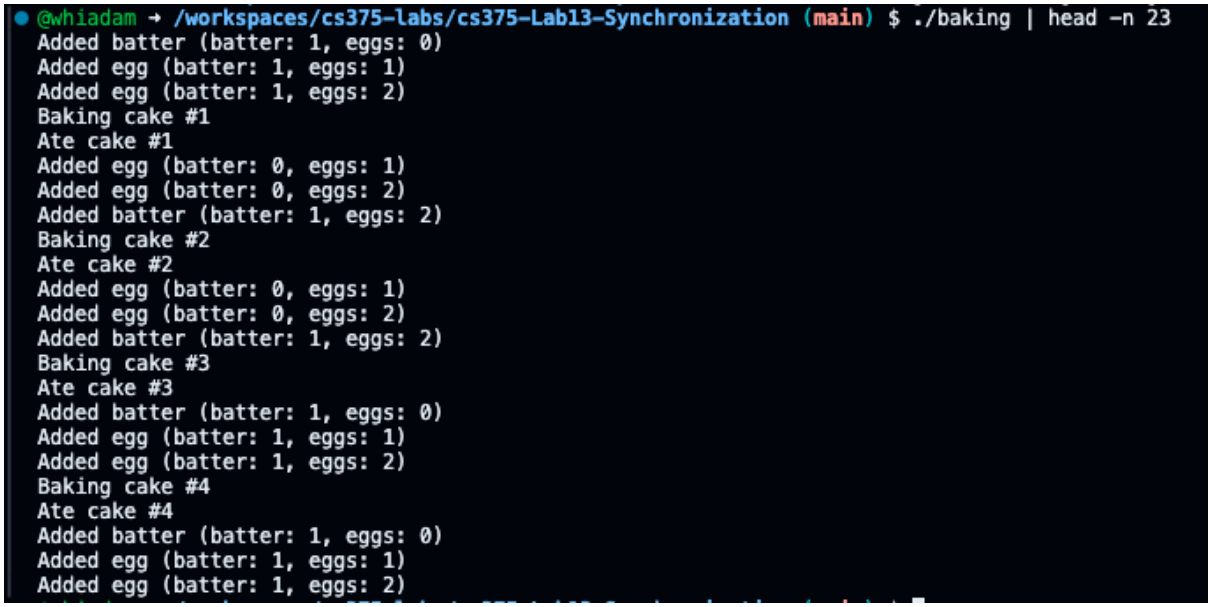


Figure 5: Compilation and execution of baking.c

7 Exercise 6: Priority Donation in Transfer

```c
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <unistd.h>

typedef struct account_t {
    pthread_mutex_t lock;
    int balance;
    long uuid;
    int priority;
} account_t;

typedef struct {
    account_t* donor;
    account_t* recipient;
    float amount;
    int priority;
} transfer_args_t;

pthread_mutex_t prio_lock = PTHREAD_MUTEX_INITIALIZER;

void lock_with_donation(account_t *acc, int thread_priority) {
    if (pthread_mutex_trylock(&acc->lock) != 0) {
        pthread_mutex_lock(&prio_lock);
        if (acc->priority < thread_priority) {
            printf("Priority␣%d␣thread␣is␣blocked␣on␣account␣%ld,␣donating␣
                priority␣to␣the␣holder␣(holder␣had␣%d)\n",
                    thread_priority, acc->uuid, acc->priority);
            acc->priority = thread_priority;
        }
        pthread_mutex_unlock(&prio_lock);
        pthread_mutex_lock(&acc->lock);
    }
    pthread_mutex_lock(&prio_lock);
    if (acc->priority < thread_priority) acc->priority = thread_priority;
    pthread_mutex_unlock(&prio_lock);
```

```
}

int effective_priority(account_t *a, account_t *b, int base) {
    pthread_mutex_lock(&prio_lock);
    int p = base;
    if (a->priority > p) p = a->priority;
    if (b->priority > p) p = b->priority;
    pthread_mutex_unlock(&prio_lock);
    return p;
}

void transfer(account_t *donor, account_t *recipient, float amount, int
    thread_priority) {
    account_t *first = (donor->uuid < recipient->uuid) ? donor : recipient;
    account_t *second = (donor->uuid < recipient->uuid) ? recipient : donor;

    lock_with_donation(first, thread_priority);
    lock_with_donation(second, thread_priority);

    printf("Priority␣%d␣thread␣holds␣accounts␣%ld␣and␣%ld\n", thread_priority,
        first->uuid, second->uuid);
    usleep(200000);
    printf("Priority␣%d␣thread␣is␣running␣at␣effective␣priority␣%d\n",
           thread_priority, effective_priority(first, second, thread_priority))
               ;

    if (donor->balance < amount) {
        printf("Insufficient␣funds.\n");
    } else {
        donor->balance -= amount;
        recipient->balance += amount;
        printf("Transferred␣%f␣from␣account␣%ld␣to␣%ld\n", amount, donor->uuid,
            recipient->uuid);
    }

    pthread_mutex_lock(&prio_lock);
    second->priority = 0; // Reset priority
    first->priority = 0;
    pthread_mutex_unlock(&prio_lock);
    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* transfer_thread(void* arg) {
    transfer_args_t *params = (transfer_args_t*)arg;
    transfer(params->donor, params->recipient, params->amount, params->priority
        );
    return NULL;
}

int main() {
    account_t acc1 = {PTHREAD_MUTEX_INITIALIZER, 1000, 1, 0};
    account_t acc2 = {PTHREAD_MUTEX_INITIALIZER, 500, 2, 0};

    transfer_args_t high = {&acc1, &acc2, 200, 2};
    transfer_args_t low = {&acc2, &acc1, 100, 1};

    pthread_t t1, t2;
    pthread_create(&t2, NULL, transfer_thread, &low);
    usleep(50000);
    pthread_create(&t1, NULL, transfer_thread, &high);

    pthread_join(t1, NULL);
```

```
    pthread_join(t2, NULL);

    printf("Final balance of account %ld: %d\n", acc1.uuid, acc1.balance);
    printf("Final balance of account %ld: %d\n", acc2.uuid, acc2.balance);
    return 0;
}
```

Explanation: Priority inversion is when a high priority thread waits on a lock held by a low priority one. When a thread gets blocked it gives its priority to the holder, and the holder resets it to 0 when its done.

Analysis: The output shows the priority 2 thread donating and the low thread running at priority 2 so it finishes faster. The accounts still lock in uuid order so there is no deadlock.

Screenshot:

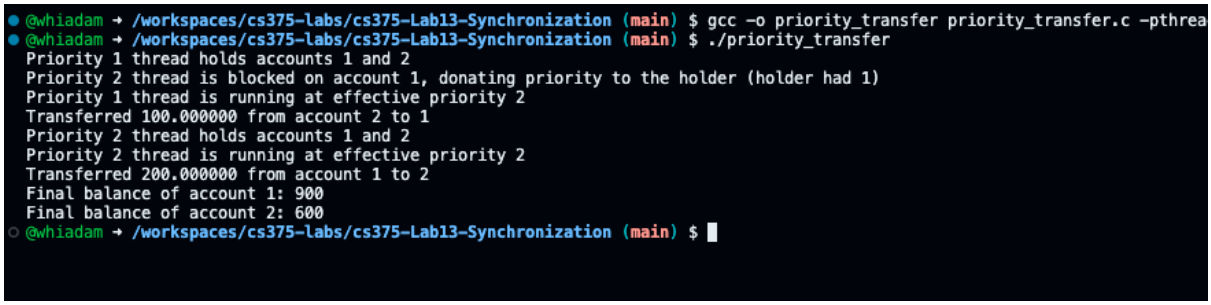


Figure 6: Compilation and execution of priority_transfer.c

8 Exercise 7: Barrier Synchronization

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define NUM_THREADS 4

pthread_mutex_t lock;
pthread_cond_t cv;
int count = 0;
int generation = 0;

void barrier() {
    pthread_mutex_lock(&lock);
    int my_generation = generation;
    count++;
    if (count < NUM_THREADS) {
        while (my_generation == generation) {
            pthread_cond_wait(&cv, &lock);
        }
    } else {
        count = 0;
        generation++;
        pthread_cond_broadcast(&cv);
    }
    pthread_mutex_unlock(&lock);
}

void* worker(void* arg) {
    int id = *(int*)arg;
    usleep((id + 1) * 200000);
    printf("Thread %d: Before barrier\n", id);
```

```
    barrier();
    printf("Thread %d: After barrier\n", id);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];
    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);
    return 0;
}
```

Explanation: Each thread adds to count and waits until the last one gets there and broadcasts. I changed the if to a while loop with a generation number so threads cant leave early.

Analysis: All the Before barrier lines print before any After barrier lines. It needs broadcast because signal only wakes one thread.

Screenshot:

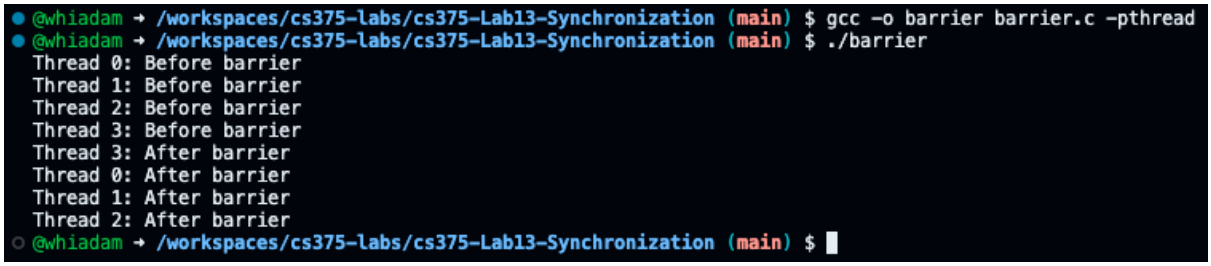


Figure 7: Compilation and execution of barrier.c

9 Exercise 8: Readers-Writers with Priority

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t lock;
pthread_cond_t reader_cv, writer_cv;
int reader_count = 0, writer_waiting = 0;
int shared_data = 0;

void* reader(void* arg) {
    int id = *(int*)arg;
    pthread_mutex_lock(&lock);
    while (writer_waiting > 0) {
        pthread_cond_wait(&reader_cv, &lock);
    }
    reader_count++;
```

```
    pthread_mutex_unlock(&lock);

    printf("Reader %d reads: %d\n", id, shared_data);
    usleep(100000);

    pthread_mutex_lock(&lock);
    reader_count--;
    if (reader_count == 0) {
        pthread_cond_broadcast(&writer_cv);
    }
    pthread_mutex_unlock(&lock);
    return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;
    pthread_mutex_lock(&lock);
    writer_waiting++;
    while (reader_count > 0) {
        pthread_cond_wait(&writer_cv, &lock);
    }
    writer_waiting--;

    shared_data++;
    printf("Writer %d writes: %d\n", id, shared_data);

    pthread_cond_broadcast(&reader_cv);
    pthread_cond_broadcast(&writer_cv);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    int reader_ids[3] = {1, 2, 3};
    int writer_ids[2] = {1, 2};
    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&reader_cv, NULL);
    pthread_cond_init(&writer_cv, NULL);

    for (int i = 0; i < 3; i++) pthread_create(&readers[i], NULL, reader, &
        reader_ids[i]);
    for (int i = 0; i < 2; i++) pthread_create(&writers[i], NULL, writer, &
        writer_ids[i]);

    for (int i = 0; i < 3; i++) pthread_join(readers[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(writers[i], NULL);

    printf("Final shared_data: %d\n", shared_data);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&reader_cv);
    pthread_cond_destroy(&writer_cv);
    return 0;
}
```

Explanation: Readers wait if a writer is waiting, so writers get to go first. I changed signal to broadcast because the second writer could get stuck.

Analysis: Multiple readers can read at once but a writer goes alone. Readers could starve if writers keep coming.

Screenshot:

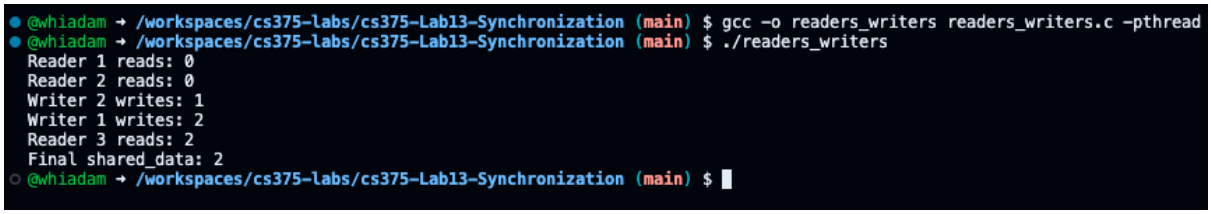


Figure 8: Compilation and execution of readers_writers.c

10 Exercise 9: Thread Pool

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NUM_THREADS 4

typedef struct {
    void (*task)(int);
    int arg;
} Task;

typedef struct {
    Task* queue;
    int head, tail, count, size;
    int shutdown;
    pthread_mutex_t lock;
    pthread_cond_t not_empty, not_full;
} ThreadSafeQueue;

void queue_init(ThreadSafeQueue* q, int size) {
    q->queue = malloc(sizeof(Task) * size);
    q->head = q->tail = q->count = 0;
    q->size = size;
    q->shutdown = 0;
    pthread_mutex_init(&q->lock, NULL);
    pthread_cond_init(&q->not_empty, NULL);
    pthread_cond_init(&q->not_full, NULL);
}

void queue_push(ThreadSafeQueue* q, Task task) {
    pthread_mutex_lock(&q->lock);
    while (q->count == q->size) {
        pthread_cond_wait(&q->not_full, &q->lock);
    }
    q->queue[q->tail] = task;
    q->tail = (q->tail + 1) % q->size;
    q->count++;
    pthread_cond_signal(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
}

int queue_pop(ThreadSafeQueue* q, Task* task) {
    pthread_mutex_lock(&q->lock);
    while (q->count == 0 && !q->shutdown) {
        pthread_cond_wait(&q->not_empty, &q->lock);
    }
    if (q->count == 0) {
        pthread_mutex_unlock(&q->lock);
        return 0;
```

```
    }
    *task = q->queue[q->head];
    q->head = (q->head + 1) % q->size;
    q->count--;
    pthread_cond_signal(&q->not_full);
    pthread_mutex_unlock(&q->lock);
    return 1;
}

void queue_shutdown(ThreadSafeQueue* q) {
    pthread_mutex_lock(&q->lock);
    q->shutdown = 1;
    pthread_cond_broadcast(&q->not_empty);
    pthread_mutex_unlock(&q->lock);
}

void queue_destroy(ThreadSafeQueue* q) {
    free(q->queue);
    pthread_mutex_destroy(&q->lock);
    pthread_cond_destroy(&q->not_empty);
    pthread_cond_destroy(&q->not_full);
}

void* worker(void* arg) {
    ThreadSafeQueue* q = (ThreadSafeQueue*)arg;
    while (1) {
        Task task;
        if (!queue_pop(q, &task)) {
            break;
        }
        task.task(task.arg);
    }
    return NULL;
}

void sample_task(int arg) {
    printf("Task executed with arg: %d\n", arg);
}

int main() {
    ThreadSafeQueue q;
    queue_init(&q, 10);
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, worker, &q);
    }

    for (int i = 0; i < 10; i++) {
        queue_push(&q, (Task){sample_task, i});
    }

    sleep(5); // Allow tasks to execute

    queue_shutdown(&q);
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    queue_destroy(&q);
    printf("All workers finished\n");
    return 0;
}
```

Explanation: 4 worker threads take tasks from a queue protected by a mutex. I made pop wait on

not_empty instead of sleeping and added a shutdown flag so main can join the workers.

Analysis: Its faster than making a new thread for every task since the same 4 threads get reused. All 10 tasks ran once.

Screenshot:

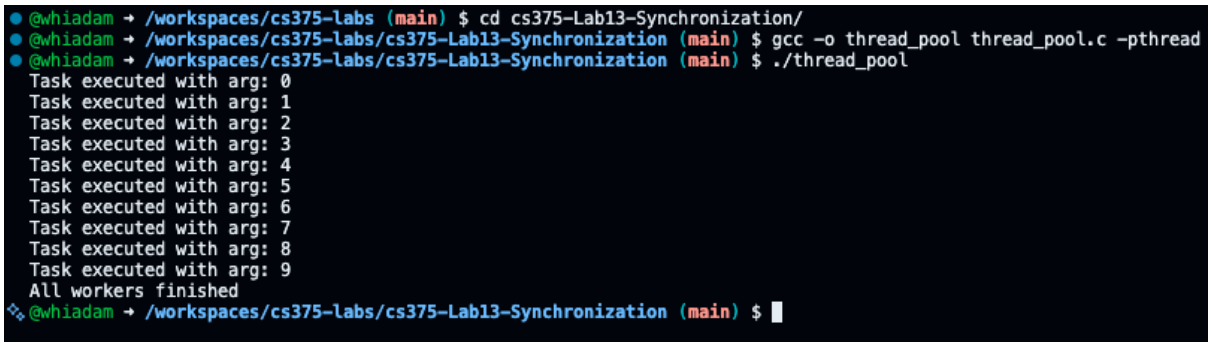


Figure 9: Compilation and execution of thread_pool.c